

PayPal Agentic AI System

Technical Documentation — FastAPI Web Edition

Architecture • Design • Implementation • API • Setup • Testing

1. Executive Summary

The PayPal Agentic AI System is a Python-based agentic application designed to translate natural-language PayPal requests into controlled API/tool operations. The latest version adds a FastAPI web layer so the system can be accessed from a browser and through an OpenAPI/Swagger interface.

The design separates user interaction, agent orchestration, domain understanding, tool retrieval, planning, confirmation, and external API execution. This separation is intended to reduce uncontrolled execution of sensitive financial actions and make the system easier to extend and test.

2. Project Objectives

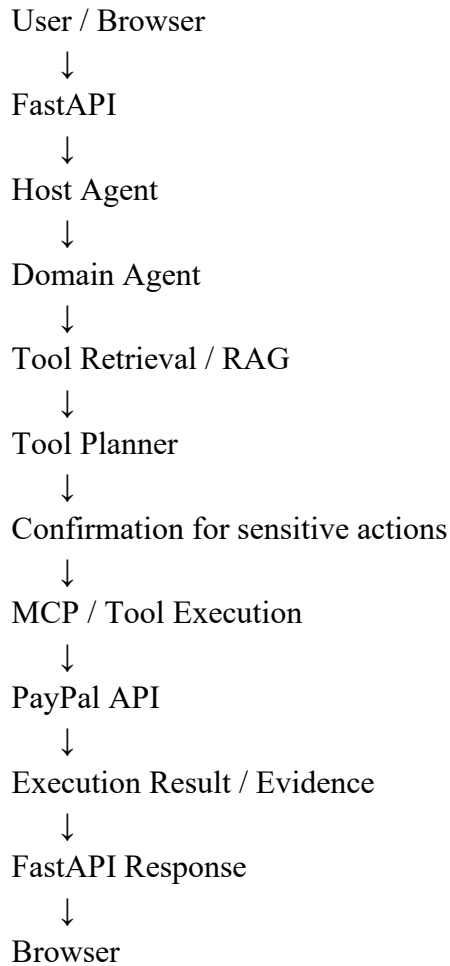
- Accept natural-language requests related to PayPal operations.
- Identify the relevant PayPal domain and capability.
- Retrieve/select an appropriate tool rather than hard-coding every request path.
- Create an execution plan with required parameters.
- Require explicit confirmation for sensitive operations where applicable.
- Execute approved operations through a controlled MCP/tool layer.
- Return structured execution results and evidence to the user.
- Expose the system through FastAPI, a browser UI, and Swagger/OpenAPI documentation.

3. Problem Statement

Directly connecting a language model to financial APIs can create ambiguity around intent, tool selection, parameters, authorization, and irreversible actions. A user may express the same business intent in many different ways, while the underlying PayPal API requires specific operations and parameters.

The project addresses this by introducing intermediate layers between natural language and external execution: domain routing, capability/tool retrieval, planning, confirmation, and controlled execution.

4. Proposed Solution



5. High-Level Architecture

The architecture follows a layered agentic pattern. FastAPI is the application boundary; the agent layers handle reasoning and orchestration; the retrieval/planning layer maps intent to an executable capability; the MCP/tool layer isolates external execution; and PayPal remains the system of record for the actual payment-domain operation.

Layer	Responsibility
Browser / Client	Sends a natural-language request and displays the result.
FastAPI	HTTP interface, validation, Swagger/OpenAPI, browser-facing endpoints.
Host Agent	Top-level orchestration and workflow coordination.
Domain Agent	PayPal-specific intent/domain handling.
Tool Retrieval / RAG	Finds relevant capabilities/tools from available definitions.
Tool Planner	Selects a tool, constructs parameters, and determines execution flow.
Confirmation	Adds an explicit approval step for sensitive operations.
MCP / Tool Layer	Provides controlled, standardized tool execution.
PayPal API	Performs the actual external PayPal operation.
Evidence / Response	Captures status, tool information, execution details, and errors.

6. Detailed Request Lifecycle

1. The user submits a natural-language request through the browser or API.
2. FastAPI validates the request and forwards it into the application workflow.
3. The Host Agent determines the overall workflow and delegates PayPal-specific understanding.
4. The Domain Agent interprets the business intent, such as payment, refund, invoice, order, or dispute handling.
5. The retrieval layer searches available tool/capability definitions for a relevant operation.
6. The planner builds a structured execution plan and determines required parameters.
7. If the planned operation is sensitive, the system pauses for explicit user confirmation.
8. After approval, the MCP/tool layer invokes the selected operation.
9. The PayPal API returns a result or an error.
10. The system packages the outcome and returns a structured response through FastAPI.

7. Agent Design

7.1 Host Agent

The Host Agent acts as the high-level coordinator. Its responsibility is to manage the overall interaction rather than directly implementing every PayPal operation.

7.2 Domain Agent

The Domain Agent provides PayPal-domain context and helps connect user intent to the appropriate business capability.

7.3 Tool Retrieval

The retrieval component reduces the need to hard-code a direct mapping for every natural-language phrase. It can locate relevant tool definitions/capabilities based on the request.

7.4 Tool Planning

The planner converts the selected capability into an execution plan, including the intended tool, parameters, and whether additional confirmation is needed.

8. Safety and Confirmation Design

A key design principle is separating reasoning from authorization to execute sensitive financial actions. Read-only requests can follow a direct information path, while actions such as refunds or captures can require explicit confirmation before execution.

Example:

"Show payment information"



Read-only operation



Execute / return information

"Refund this payment"



Sensitive operation detected



Confirmation required



User confirms



Execute refund tool

This pattern is important because an LLM should not be treated as an unrestricted authority to perform financial operations.

9. FastAPI Web Architecture

The latest project version adds FastAPI as the web/API entry point. This allows the same agentic workflow to be exercised through a browser and through machine-readable HTTP APIs.

Endpoint / Component	Purpose
GET /	Browser-facing interface for submitting a request.
POST /api/ask	Accepts a natural-language request and invokes the agent workflow.
GET /health	Basic application health check.
GET /docs	FastAPI-generated Swagger/OpenAPI interface.
GET /redoc	FastAPI-generated ReDoc API documentation.

10. Browser Interaction

The browser layer provides a simple way to submit a request without requiring a command-line client. The user enters a PayPal-related question or operation, submits it, and receives the agent response and relevant execution information.

Browser

→ <http://127.0.0.1:8000/>

API documentation

→ <http://127.0.0.1:8000/docs>

Alternative API documentation

→ <http://127.0.0.1:8000/redoc>

11. API Usage Example

Example request to the agent endpoint:

POST /api/ask

Content-Type: application/json

```
{
  "query": "What tools are available for managing invoices?"
}
```

A typical response is structured around the agent result and execution metadata, for example:

```
{
  "answer": "...",
  "status": "...",
  "tool_used": "...",
  "execution_id": "...",
  "plan": [...]
}
```

The exact response fields depend on the implementation in the supplied project and should be treated as the application contract rather than a generic FastAPI response.

12. Project Structure

.env/

.env.example/

.gitignore/

ARCHITECTURE.md/

README.md/

RUNNING.md/

app/

- app/__init__.py
- app/agents/__init__.py
- app/agents/domain_agent.py
- app/agents/host_agent.py
- app/agents/models.py
- app/capabilities/__init__.py
- app/capabilities/rag.py
- app/capabilities/system_search.py
- app/config.py
- app/llm.py
- app/mcp/__init__.py
- app/mcp/client.py
- app/mcp/server.py
- app/observability.py
- app/recovery/__init__.py
- app/recovery/handler.py
- app/state/__init__.py
- app/state/state.py
- app/system.py
- app/templates/index.html
- app/tools/__init__.py
- app/tools/credentials.py
- app/tools/executor.py
- app/tools/models.py
- app/tools/postman_parser.py

- app/tools/registry.py
- app/tools/retriever.py
- app/tools/schema.py
- app/web.py
- app/workflows/__init__.py
- app/workflows/graph.py

data/

- data/Datazoic Programming Task (1).pdf
- data/PayPal APIs.postman_collection.json
- data/knowledge/disputes.md
- data/knowledge/errors_and_limits.md
- data/knowledge/invoicing.md
- data/knowledge/payment_lifecycle.md
- data/tool_embeddings.npz
- data/tool_registry.json

pyproject.toml/

requirements.txt/

run_smoke.bat/

scripts/

- scripts/smoke.py

setup_windows.bat/

setup_windows.ps1/

tests/

- tests/test_capabilities.py
- tests/test_domain_agent.py
- tests/test_executor.py
- tests/test_host_agent.py
- tests/test_mcp.py
- tests/test_postman_parser.py
- tests/test_recovery.py
- tests/test_registry.py
- tests/test_retriever.py
- tests/test_workflow.py

13. Key Technologies

Technology	Role
Python	Core application language.
FastAPI	Web/API layer and OpenAPI documentation.
Uvicorn	ASGI server used to run FastAPI.
Agentic workflow	Intent understanding, orchestration, planning and execution.
RAG / retrieval	Capability/tool discovery.
MCP / tool layer	Controlled external tool execution.
PayPal APIs	External payment-domain operations.
Pytest	Automated testing.

14. Installation and Environment Setup

11. Install Python 3.11 or a compatible project-supported version.
12. Extract the ZIP file.
13. Open a terminal in the project root.
14. Create a virtual environment.
15. Activate the virtual environment.
16. Install dependencies from requirements.txt.
17. Create/configure the .env file using the provided example.
18. Add the required API credentials securely.

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
```

15. Running the Application

Start the FastAPI server with:

```
python -m uvicorn app.web:app --host 127.0.0.1 --port 8000 --reload
```

Then open the browser at:

```
http://127.0.0.1:8000/
```

For interactive API testing:

<http://127.0.0.1:8000/docs>

16. Testing Strategy

The project contains automated tests covering important agentic components. The recommended test sequence is to install all dependencies first and then execute the suite from the project root.

```
pytest -q
```

A smoke-test workflow can also be used to exercise representative agent scenarios. Tests should be run in an environment where all project dependencies are installed and network/API credentials are configured when an integration test requires them.

17. Error Handling and Recovery

- Input and API-level validation should reject malformed requests before execution.
- Tool selection and planning should surface missing or ambiguous parameters.
- Sensitive actions should not execute before required confirmation.
- External PayPal errors should be captured and returned as structured application errors.
- Execution metadata should be retained where supported so failures can be diagnosed.
- Integration tests should distinguish mocked/sandbox behavior from production PayPal behavior.

18. Security Considerations

- Keep PayPal client credentials and LLM API keys outside source control.
- Use environment variables or a proper secrets manager for credentials.
- Use PayPal sandbox credentials during development and testing.
- Do not allow the LLM to bypass the confirmation policy for sensitive actions.
- Validate tool parameters before execution.
- Restrict production credentials and endpoints to controlled deployment environments.
- Avoid logging sensitive payment data or credentials.

19. Design Rationale

Design Choice	Reason
FastAPI boundary	Provides a clean HTTP interface and automatic API documentation.
Separate agents	Separates orchestration from PayPal-domain reasoning.
Retrieval	Supports extensible capability discovery instead of a large hard-coded mapping.
Planner	Makes the intended operation and parameters explicit before execution.
Confirmation	Adds a human-approval boundary for sensitive financial operations.
MCP/tool layer	Separates model reasoning from external execution.
Sandbox-first development	Reduces risk while developing PayPal integrations.

20. Example End-to-End Scenarios

Scenario A — Information Request

User: "What tools are available for managing invoices?"

FastAPI
↓
Host Agent
↓
Domain Agent
↓
Tool Retrieval
↓
Tool information returned
↓
FastAPI
↓
Browser

Scenario B — Sensitive Operation

User: "Refund the payment for ORDER-1"

FastAPI
↓
Host Agent
↓
Domain Agent
↓
Tool Retrieval
↓
Planner
↓
Confirmation required
↓
User confirmation
↓
MCP / Tool
↓
PayPal
↓
Execution result
↓
Browser

21. Extensibility

The architecture is intended to make new PayPal capabilities easier to add. A new operation can be represented as a tool/capability with a clear schema and execution contract. The retrieval layer can then make that capability discoverable, while the planner and confirmation policy determine how it should be executed.

- Add additional PayPal payment capabilities.
- Add more invoice and dispute workflows.
- Add stronger parameter validation.
- Add persistent conversation/session state.
- Add authentication and role-based access control.
- Add observability and tracing.
- Deploy the FastAPI application behind a production ASGI server/reverse proxy.
- Add CI/CD for automated testing and deployment.

22. Limitations

- Agentic behavior depends on the quality of tool definitions, retrieval, prompts, and model output.
- External PayPal behavior depends on sandbox/production API availability and credentials.
- A local development server is not a production deployment architecture.
- Sensitive operations require careful authorization, auditing, and secrets management before production use.
- The current browser UI is an interface for the agentic workflow, not a replacement for PayPal's own security and compliance controls.

23. Conclusion

The latest ZIP represents a layered agentic architecture with a FastAPI web/API interface. Its core design focuses on controlled translation from natural language to external PayPal operations, with retrieval, planning, and confirmation separating model reasoning from financial execution. This structure provides a clear basis for testing, extending, and demonstrating the system.

Appendix A — Supplied Project Files

- .env
- .env.example
- .gitignore
- ARCHITECTURE.md
- README.md
- RUNNING.md
- app/__init__.py
- app/agents/__init__.py
- app/agents/domain_agent.py
- app/agents/host_agent.py
- app/agents/models.py
- app/capabilities/__init__.py
- app/capabilities/rag.py
- app/capabilities/system_search.py
- app/config.py
- app/llm.py
- app/mcp/__init__.py
- app/mcp/client.py
- app/mcp/server.py
- app/observability.py
- app/recovery/__init__.py
- app/recovery/handler.py
- app/state/__init__.py
- app/state/state.py
- app/system.py
- app/templates/index.html
- app/tools/__init__.py
- app/tools/credentials.py
- app/tools/executor.py
- app/tools/models.py
- app/tools/postman_parser.py
- app/tools/registry.py
- app/tools/retriever.py
- app/tools/schema.py
- app/web.py
- app/workflows/__init__.py
- app/workflows/graph.py
- data/Datazoic Programming Task (1).pdf
- data/PayPal APIs.postman_collection.json
- data/knowledge/disputes.md

- data/knowledge/errors_and_limits.md
- data/knowledge/invoicing.md
- data/knowledge/payment_lifecycle.md
- data/tool_embeddings.npz
- data/tool_registry.json
- pyproject.toml
- requirements.txt
- run_smoke.bat
- scripts/smoke.py
- setup_windows.bat
- setup_windows.ps1
- tests/test_capabilities.py
- tests/test_domain_agent.py
- tests/test_executor.py
- tests/test_host_agent.py
- tests/test_mcp.py
- tests/test_postman_parser.py
- tests/test_recovery.py
- tests/test_registry.py
- tests/test_retriever.py
- tests/test_workflow.py